Parametric Formula Student Spaceframe: Generator, Beam FE Solver and Stiffness Optimisation

Satt Gupta — personal project, Summer 2026

1. Problem

A Formula Student chassis has to be stiff in torsion. The chassis sits between the front and rear suspension, so if it twists under cornering loads it behaves as an extra spring in series with the springs and anti-roll bars. Load that was meant to go to one wheel is absorbed by the frame flexing instead, and the suspension setup stops behaving predictably. Published Formula Student targets commonly sit around 1500–2000 N·m/deg.

Stiffness is cheap to add badly and expensive to add well. A team can always weld in more steel, but mass costs lap time everywhere. The useful question is not "how do we make it stiffer" but "where is the next gram of steel worth spending", and that question needs a model.

I set out to build the whole chain myself: generate a rules-legal frame from parameters, analyse it with a finite element solver I wrote from scratch, and search tube sizes for the best torsional stiffness per kilogram.

2. What I built

Generator. A Python script that builds a complete steel spaceframe from a handful of parameters — hoop positions, car width, which tubes go where — and writes the geometry out as JSON. The Formula Student T3 rules on tube sizing and geometry are encoded as constraints inside the generator, so an illegal frame cannot be produced rather than being caught afterwards. The output frame has 36 nodes and 81 members and weighs 34.18 kg.

Three decisions shaped it. Geometry is built from positive gaps rather than absolute stations, which makes ordering errors unrepresentable. The rule checker never reads the input parameters — it sees only nodes and members and re-derives every quantity from coordinates, so it cannot simply confirm that the clamps ran. And tubes are drawn from a catalogue of purchasable sizes, with the legaliser picking the lightest one that meets each rule class.

Solver. A 3D beam finite element solver written from scratch in NumPy. Each tube becomes a 12 × 12 element stiffness matrix covering axial, torsional and biaxial bending behaviour, rotated into car axes and assembled into a 216 × 216 global matrix. Boundary conditions are applied by deleting rows and columns, and the system is solved for displacement.

I used beam elements rather than a truss because the joints are welded and therefore transfer moments. This was not an assumption I made on faith: as pin-jointed bars the frame has roughly 21 mechanisms and a truss solver returns a singular matrix, and measured on the real solution, 69 of the 81 members carry more bending stress than axial stress, with a median bending-to-axial ratio of 8.5.

Load cases. Torsion is the headline — a ±500 N couple at the four front suspension pickups, giving 408.6 N·m over the 408.6 mm lateral spacing of the loaded nodes. Bending (an 80 kg driver and 70 kg powertrain at 1 g) and a 10 kN rearward bulkhead push are also run.

Post-processing. Member forces, a combined axial-plus-bending stress check against welded yield strength, Euler and Johnson buckling checks, and a strain energy breakdown showing where the frame is actually flexing.

Sweep and optimiser. The generator is imported as a module, so a candidate frame costs about 25 ms to build in memory. The sweep changes one tube group at a time so each difference is attributable; the optimiser combines changes and re-solves the whole frame each time, because the changes interact.

3. How I checked it

Validation came from four independent directions, in descending order of how much weight they carry.

Closed-form beam solutions. The solver was built in ten blocks, each with its own tests comparing against hand formulas: cantilever tip deflection $PL^3/3EI$, tip rotation ML/EI, simply supported centre load $PL^3/48EI$, and torsion TL/GJ. This is conclusive rather than indicative, because for loads applied at nodes a beam element is not an approximation — it returns the exact textbook answer. Agreement to about 1×10^{-10} therefore means correct, not close, and any larger difference would be a bug rather than "finite elements being approximate". I also confirmed that an unsupported element has exactly six zero-energy rigid-body modes, and that axial, bending and torsional behaviour is

correct for beams pointing in 20 random 3D directions.

Independent solver cross-check. I rebuilt the entire chassis in PyNite, a mature third-party open-source 3D frame solver with its own element formulation, and compared. The two agree to about 1×10^{-12}. Two independently written programs agreeing to twelve decimal places on a 216-degree-of- freedom problem is very unlikely if either is wrong, and unlike the closed-form tests this exercises the assembly, the coordinate rotations, the boundary conditions and the solve rather than just the element matrices. This is the strongest single piece of evidence in the project.

Physical identities. Checks that hold where no closed form exists: support reactions balance the applied loads per axis; total member strain energy equals the work done by the loads to about 1×10^{-13}; tripling the load triples every displacement exactly; mirroring the frame in y leaves the vertical answer unchanged, which catches handedness and sign errors in the 3D rotation; and Maxwell–Betti reciprocity holds.

Regression pins and plausibility. The frame must keep returning 1712.2 N·m/deg, and the same frame with its axle bracing removed 953.9 N·m/deg. Every one of roughly 1700 swept candidates is re-run through all 52 rule checks, with zero failures. The baseline sits inside the range of published Formula Student chassis figures, which is a plausibility check rather than validation, since published cars differ in load points and supports.

4. Results

Baseline frame: **1712.2 N·m/deg at 34.18 kg**, or 50.1 N·m/deg per kg.

Configuration Stiffness Mass Per kg Baseline 1712 N·m/deg 34.18 kg 50.1 After dominance pass 1956 N·m/deg 34.08 kg 57.4 Greedy search, 2 kg budget 2499 N·m/deg 35.97 kg 69.5 Greedy search, 4 kg budget 3141 N·m/deg 38.06 kg 82.5

Every design above is rules-legal, and the worst buckling utilisation *falls* from 0.108 to 0.055 along the way, so the stiffness is not bought by running members closer to their limits.

About 14% more stiffness is available at **no mass cost at all**, by swapping to wider, thinner tubes. A 25 × 2.5 mm and a 30 × 2.0 mm tube have the same mass, but the second has 53% more second moment of area, because I grows with roughly the fourth power of diameter while mass grows only with cross-sectional area. On the main hoop alone that single swap is worth +100 N·m/deg for 17 g *less* mass. The rules permit it because Table 4 specifies minimum wall thickness, area and second moment — not a diameter.

The result I learned most from was a wrong one. My first run returned 954 N·m/deg. Nothing looked obviously broken: no absurd displacements, and the solver was already validated against closed-form solutions. My instinct was that the tubes were too small, which would have been the wrong fix. Instead I added a strain energy breakdown per member, and found 44% of the frame's elastic energy sitting in two thin bulkhead-support tubes. The suspension load was entering at nodes part-way along unbraced tubes, so the number I was measuring was one tube bending, not the chassis twisting. I triangulated the axle station with five tubes so the load lands on a braced node; stiffness rose 80% to 1712 N·m/deg and the energy share in those groups fell from 44% to 8%.

The lesson is the part worth keeping: where you apply the load is a modelling decision, and getting it wrong invalidates the result while the solver reports no error at all. No amount of tube resizing would have found it.

5. Limitations

The model is linear, static and small-deflection, using Euler–Bernoulli beams with no shear deformation. It is invalid above yield and useless for impact.

Joints are modelled as points. Tube diameter at a node, weld fillets and joint overlap are all absent. This matters more as the optimiser selects larger tubes — a 44.5 mm tube meeting others at a node is not the same structure as a point connection.

Nothing has been checked against a physical frame. Published work suggests built chassis typically measure 10–20% softer than their models, mostly because of joint flexibility and rig compliance, so I quote these numbers as predictions with that band in mind.

The search only changes tube sizes. Geometry and topology are fixed. The largest single gain I found anywhere was adding one diagonal above the harness bar, worth +36% stiffness for 687 g — about twice as much per gram as the entire tube-size optimisation — and the optimiser can never find it, because it is a geometry change. That diagonal also crosses the zone behind the driver's head, so whether it can actually go there is a packaging decision the solver can inform but not make.

Buckling is checked member by member from a linear solve, with no global eigenvalue buckling and no imperfection sensitivity. There is no fatigue assessment, no weld or joint failure model, and no crash loading. Hoops are modelled as sharp-cornered polylines rather than bent tubes.

Finally, the output is **rules-informed and verified against section T3, not scrutineering-legal**. Cockpit templates, driver clearance and SES approval are explicitly out of scope. A frame could pass every check here and still be undriveable.

6. Reproducing this

The repository contains 45 tests across ten blocks, each stating as a hand-checkable formula what the layer beneath it must do. Running `for f in tests/test_block*.py; do python $f; done` executes all of them. `tools/crosscheck_pynite.py` reproduces the independent comparison. Every figure in `results/` is regenerated by the entry-point scripts, and `case_studies/run_case_studies.py` prints the numbers behind all four case studies.